UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

Applied Engineering Optimisation

ENEL445

Week 6: Optimisation-based control

- Introduction to optimal control
- Key principles
- Predictive control
- Dynamic programming
- Reinforcement learning

Real-life applications

- **Control**: Most controllers in industry are proportional-integral, but model-predictive control (MPC) approaches are increasingly common.
- **Dynamic Programming** is widely used as a tool to solve computationally difficult problems – e.g. in 2015 Win-And-Score-Predictor (UC) cricket predictor. It can also be used for:
- **Reinforcement Learning:** This is actually a form of optimal control -> lots of interest and increasing number of real-life applications.

Optimal control

- The main idea is to design control inputs that minimise a cost function which is based on our states from time 0 to the end of the prediction horizon (or infinity).
- This means optimization across multiple time periods.

Horizon

- In some cases, we can consider the time period between now and infinity as the system converges to a point where the cost is zero.
  - Example: LQR control (if you're doing ENME403)
- However, in many practical cases this is not possible, approximate by choosing a distant horizon which recedes with time.
  - This is called *receding horizon control* – another name for *model predictive control*
  - *The length of the horizon is a trade off between complexity and performance.*

Mathematical preliminaries

States and Inputs

State $x \in X$ (e.g. $X = \mathbb{R}^n$)

Input $u \in U$ (e.g. $U = \mathbb{R}^m$)

Dynamics

Discrete-time state space system:

$$x_{k+1} = f(x_k, u_k), \quad \text{where } f(\cdot, \cdot) : X \times U \to X$$

Assume the initial condition x_0 given.

Trajectory

Given x_0, each input sequence $u_0, u_1, \ldots, u_{h-1}$ generates a state sequence $x_0, x_1, \ldots, x_h$ starting at x_0 and such that $x_{k+1} = f(x_k, u_k)$ for $k = 0, 1, \ldots, h-1$.

Mathematical description

Finite Horizon Cost Function

$$J(\underbrace{x_0}_{\text{Initial condition}}, \underbrace{u_0,\, u_1,\, \ldots,\, u_{h-1}}_{\text{Inputs}}) = \sum_{k=0}^{h-1} \underbrace{c(x_k, u_k)}_{\text{stage cost}} + \overbrace{J_h(x_h)}^{\text{terminal cost}}$$

Objective

Find the "best" input sequence $u_0^*,\, u_1^*,\, \ldots, u_{h-1}^*$, such that

$$\begin{aligned} J^*(x_0) &= J(x_0,\, u_0^*,\, u_1^*, \ldots,\, u_{h-1}^*) \\ &= \min_{u_0, u_1, \ldots, u_{h-1}} J(x_0,\, u_0,\, \ldots,\, u_{h-1}) \end{aligned}$$

Model Predictive Control

- Direct application of constrained optimisation:

Model-predictive control

The main advantages are:

- An optimum trajectory is followed, unlike PI control which often overshoots, etc.
- Tends to work very well in practice even with short horizons.
- Constraints can be directly included in the optimization problem, e.g. x_1 has to stay between 0 and 10 which is great for ensuring physical constraints are not violated (another issue with PI control).

Model-predictive control

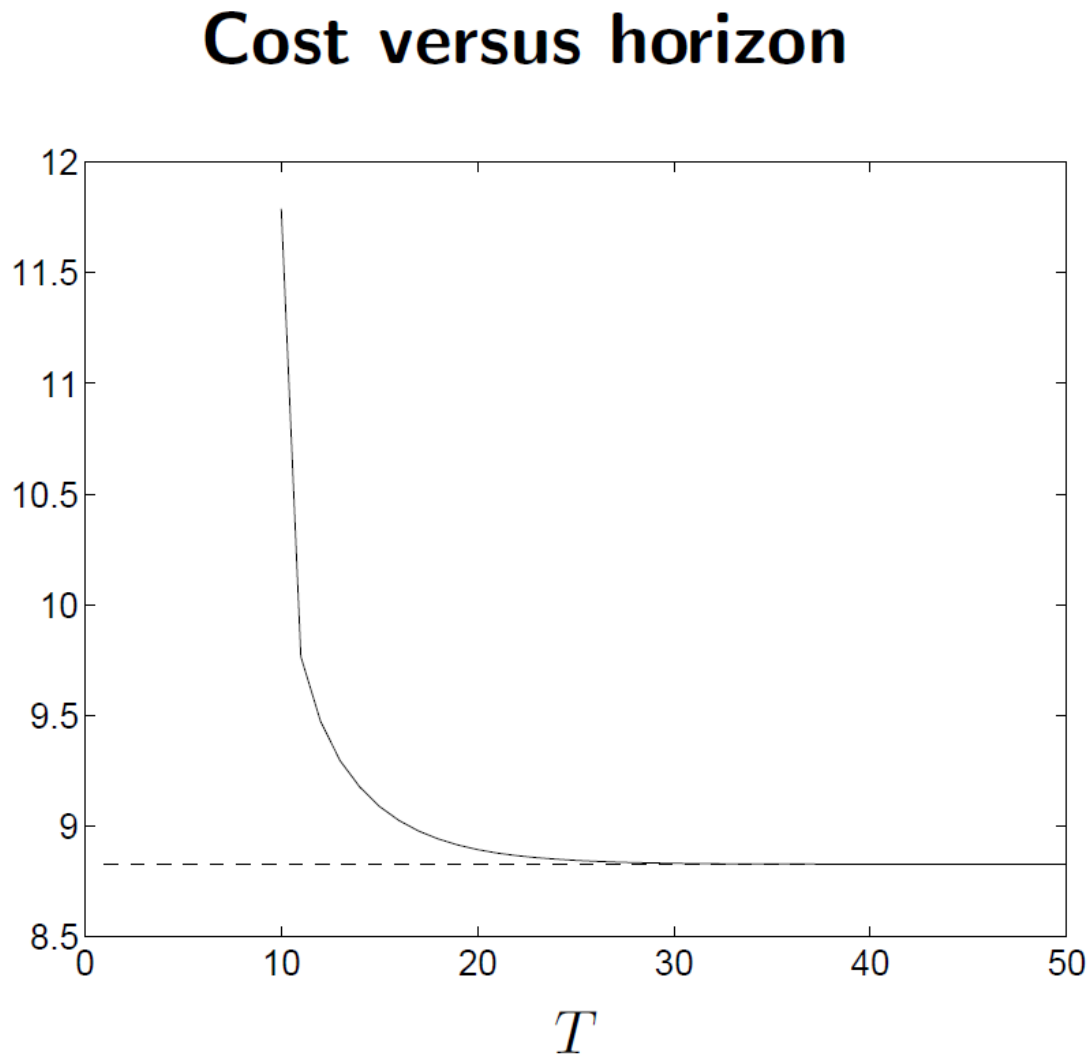
The main disadvantages are:

- An accurate model is required, since this is used for predictive purposes. Bad model = bad control and could lead even to constraints not being satisfied.
- Complex and computationally expensive (especially as the horizon *h* becomes larger), and generally quite hard to prove stability.

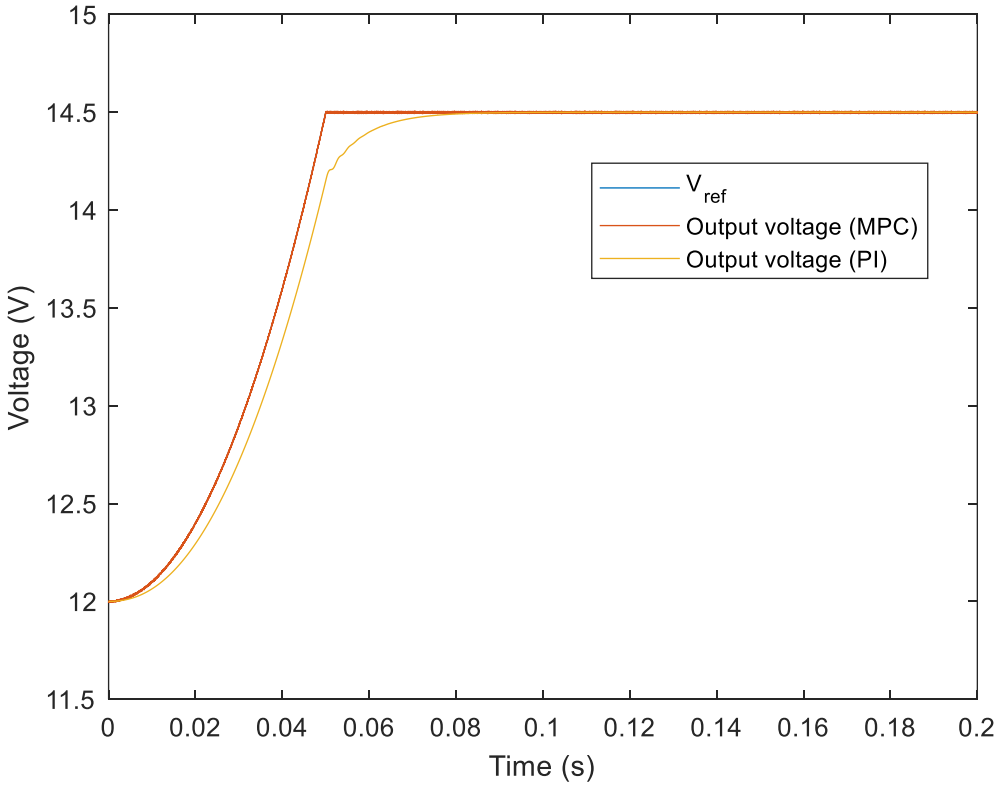
Finite horizon approximation (1)

- In practice, we cannot evaluate an infinite horizon!
- Idea 1: approximate with a finite horizon
- Example 1
  - system with $n = 3$ states, $m = 2$ inputs; A, B chosen randomly
  - quadratic stage cost: $c(x,u) = x^2 + u^2$
  - $\mathcal{X} = \{ x \mid \| x \|_\infty \leq 1 \}$, $\mathcal{U} = \{ u \mid \| u \|_\infty \leq 0.5 \}$
  - initial point: $z = (0.9, -0.9, 0.9)$
  - optimal cost is $V(z) = 8.83$

Example 1



Example 2: Buck converter



Code on Learn: $h = 2$, $Ts = 10^{-4}$ s

Finite horizon approximation (2)

- Idea 2: solve MPC problem every *K* steps, use plan for *K* steps, then solve another MPC problem, etc.

- Variations:
  - add estimated final state cost $\hat{V}(x(t+T))$ to truncate horizon.
    - If $\hat{V} = V$, MPC solution is the optimal solution.
  - convert hard constraints to violation penalties – same idea as penalty methods in constrained optimisation!
    - Why might this be a good idea? (Hint: think about disturbances)

Bellman's Principle of Optimality

How to find the optimal sequence?

$$\min_{u_0, u_1, \ldots, u_{h-1}} \left(\sum_{i=0}^{h-1} c(x_i, u_i) + J_h(x_h) \right).$$

Even without constraints, intractable for large *h*

Fundamental idea: Bellman's Principle of Optimality

Assume that the optimal controls $u_0^*, u_1^*, \ldots, u_{h-1}^*$ lead us from x_0 to x_k at step k. Then the truncated sequence $u_k^*, \ldots, u_{h-1}^*$ is a solution for the truncated problem

$$\min_{u_k, u_{k+1}, \ldots, u_{h-1}} \left(\sum_{i=k}^{h-1} c(x_i, u_i) + J_h(x_h) \right).$$

Bellman's principle

- The value of this is that it allows us to work **backwards** to find the optimal solution.

Example

Let $U = \{1, 2, 3\}$ and $X = \{1, 2, 3\}$ and

$$x_{k+1} = u_k$$
$$c(x, u) = C_{xu}$$
$$J_h(x) = x^2$$
$$h = 5$$

$$C_{xu} = \begin{array}{c|ccc} x \backslash u & 1 & 2 & 3 \\ \hline 1 & 4 & 2 & 5 \\ 2 & 4 & 5 & 3 \\ 3 & 4 & 2 & 1 \end{array}$$

Example

Discrete-time optimal control

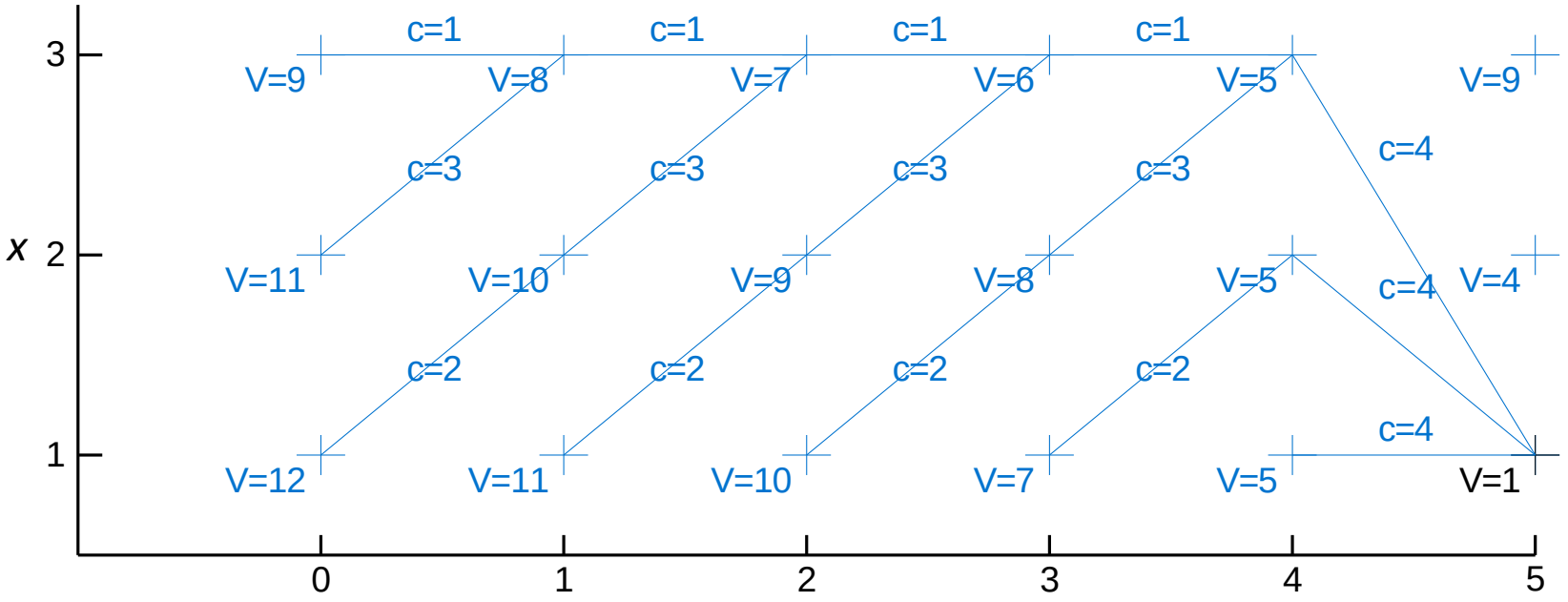
an introduction to the use of Dynamic Programming

$$x_{k+1} = u_k \qquad c(x,u) = C_{xu}$$

$$J_h(x) = x^2 \qquad h = 5$$

$C_{xu} =$

$x\backslash u$	1	2	3
1	4	2	5
2	4	5	3
3	4	2	1



Value function

- It is clear from this example that a *value function* *V(x,k)* exists, i.e. the cost of being at state *x* at time *k*.
- If we knew this function exactly, we could work out the optimal next action easily.
- However, this is dependent on a finite horizon *h* and a well-defined final cost $J_h(x,h)$. Also, what if the horizon is infinite?

Discount factors

- We could truncate (as in MPC) but often a better approach is to use a *discount factor.*

$$J(x_0) = \sum_{k=0}^{\infty} \lambda^k c(x_k, u_k)$$

- An interpretation of this is that an immediate reward is better than the same reward tomorrow, but not by that much.

Value iterations

- For such problems there is no time dependence, so the Bellman optimality condition is that

$$V(x) = \min_u (c(x, u) + \lambda V(f(x, u)))$$

- We don't know the true value function *V*, but this can be solved by **value iteration**

$$V_{k+1}(x) = \min_u (c(x, u) + \lambda V_k(f(x, u)))$$

- We start with an initial guess V_0 and then iterate, repeatedly computing $V_{k+1}(x)$ for all states *x* until *V* converges.

Example

$c(x, u)$

$x \backslash u$	1	2	3
1	5	6	4
2	4	5	3
3	4	3	2

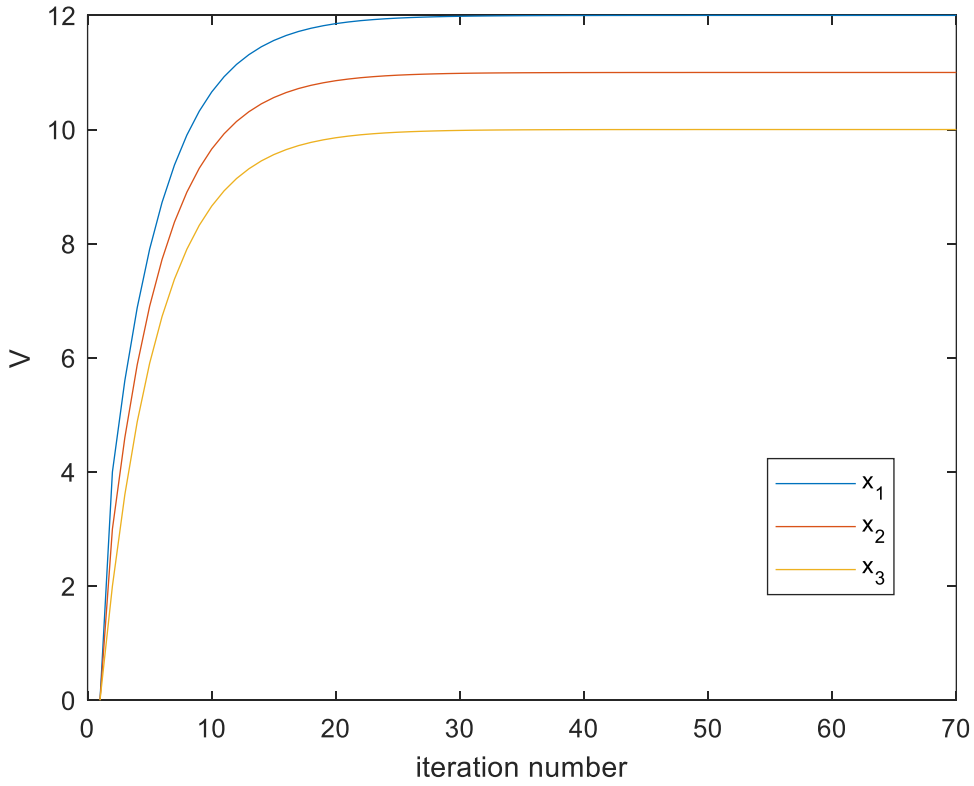
$f(x, u)$

$x \backslash u$	1	2	3
1	3	3	3
2	1	3	3
3	2	3	3

Exercise:

- If the discount factor is 0.8, solve for *V(x)* via value iterations.
- Show that the theoretical value is $V(x) = [12\ 11\ 10]^T$

Example



Code on Learn

Rewards

- Instead of a cost *c(x, u)* we can instead think of maximising a discount sum of rewards *r(s,a)* by choosing actions *a.*

$$V(s) = \max_a (r(s, a) + \lambda V(f(s, a)))$$

- Usually reinforcement learning problems are posed this way.

Policy iteration (1)

- Now consider the case where we have a policy
$$a = \pi(s)$$
- The value of that policy at state *s* is now
$$V^{\pi}(s) = r(s,a) + \lambda V^{\pi}(f(s,\pi(s))$$
- This can be computed as the limit of
$$V^{\pi}_{k+1}(s) = r(s,a) + \lambda V^{\pi}_{k}(f(s,\pi(s))$$

Policy iteration (2)

1. Initialise policy $\pi(s)$
2. Compute value of this policy V^{π} using the iterative approach on the previous slide.
3. Update $\pi(s)$ to be the **greedy** policy with respect to V^{π}

$$\pi(s) \leftarrow \arg\max_{a} V^{\pi}(f(s,a))$$

4. Stop if policy has converged, otherwise return to Step 2.

Policy iteration (3)

- This process guarantees that each policy is a strict improvement over the previous one.
- Each policy evaluation is an iterative process, but by starting with the value function from the previous policy, typically convergence speed is increased.
- Policy iteration ultimately leads to the same outcome as value iteration, given that if you have the value function you can compute the optimal policy. But, in many cases it converges more quickly.

Markov decision processes

- In a Markov decision process, the transitions between states are random. We can easily amend the previous value function definitions for this case.

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r|s, a)(r + \lambda V^{\pi}(s'))$$

Learning from samples (1)

- In theory one could learn the value function from samples of *s, a,* and *r.* The optimal action is then

$$a^*(s) = \arg\max_a \left(r(s,a) + \lambda V(f(s,a)) \right)$$

in the deterministic case and, for an MDP:

$$a^*(s) = \arg\max_a \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) \left(r(s,a) + \lambda V(s') \right)$$

Learning from samples (2)

- This is not ideal – we need to know the model and the reward function to recover the optimal action even if we have worked out the value function.
- Better idea: learn the action-value function *Q(s,a)*

$$Q(s,a) = r(s,a) + \lambda V(f(s,a))$$

Or

$$Q(s,a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r|s, a)(r + \lambda V(s'))$$

Learning from samples (3)

- The value function is easily obtained from Q

$$V(s) = \max_a Q(s, a)$$

- So the Q function satisfies the recursion (deterministic case):

$$Q(s, a) = r(s, a) + \lambda \max_b Q(f(s, a), b)$$

Monte Carlo exploration (basic idea)

1. Choose a random start point and action, simulate a trajectory (where all actions except the starting one are greedy actions w.r.t. the current estimate of Q).
2. Calculate reward from the trajectory.
3. Update estimate of Q based on this reward.
4. Repeat 1 until convergence.

- This algorithm is the basis of many current approaches to reinforcement learning (e.g. AlphaZero etc). Many people believe that this algorithm always converges to the optimal action-value function, but it has never been proved.

Q learning (1)

- Unlike Monte-Carlo methods which update after complete trajectory has been sampled, Q learning updates for every sample.
- Given samples *s, a, r, s'*, update:

$$Q_{k+1}(s,a) = (1-\alpha_k)Q_k(s,a) + \alpha_k(r(s,a) + \lambda \max_b Q(s',b)$$

- α_k is the *learning rate* and is decayed to zero gradually.

Q learning (2)

- In order to get samples, a policy is needed to choose actions *a.*
- Q learning is an *off-policy* method. It learns the value of the optimal policy whilst sampling using *any* policy. One common approach is the ε-greedy policy:

$$\pi(s) = \begin{cases} \arg\max Q(s, a), & \text{with probability } 1 - \epsilon \\ \text{randomly selected}, & \text{with probability } \epsilon \end{cases}$$

Continuous action/state-space

- If *s* is continuous then a functional approximation for *Q(s, a)* is required which must allow previously visited states to be interpolated, such as a neural network.
- If the action space is also continuous then a functional approximation can still be used as long as action of minimising *Q(s, a)* over *a* is feasible.
  - Otherwise an actor-critic method can be employed - where a second network is used to return an action given a state (i.e. to represent a policy) and the policy network and Q network updated independently.

Summary: What method to choose?

1. Dynamic Programming: Requires low state dimension, small number of inputs and a model. Particularly useful if the end point is known, as can then reverse time from there.
2. Predictive Control: Effective way of dealing with constraints, but the n-step ahead optimisation needs to be able to be done efficiently, e.g. linear model, quadratic cost + constraints.
3. Q-learning: Useful if no model is available and complex nonlinear control solutions are required. Can be implemented directly for low dimensional problems, otherwise needs functional approximations for the Q function and, possibly, the policy (e.g. if the action and/or state-space are continuous).